

Grumpy Bookstore Owner

Epam Amazon Nutanix

Authored by **Arpit Raj**
 LNMIIT Jaipur • June 29, 2026

Problem Overview & Visual Tracing

The problem asks us to maximize the number of satisfied customers throughout a day. We are given a customers array and a binary grumpy array where 1 denotes that the owner is grumpy (and customers become unsatisfied) and 0 denotes the owner is calm (customers are naturally satisfied). We are also granted a special technique to remain calm for a continuous block of *minutes*.

Visual Example Execution (From Notes)

Consider the scenario provided in the handwritten analysis:

- **Customers Array:** [1, 0, 1, 2, 1, 1, 7, 5]
- **Grumpy Array:** [0, 1, 0, 1, 0, 1, 0, 1]

Index (i)	0	1	2	3	4	5	6	7
Customers	1	0	1	2	1	1	7	5
Grumpy	0	1	0	1	0	1	0	1
Base Status	Satisfied (1)	Grumpy (0)	Satisfied (1)	Grumpy (0)	Satisfied (1)	Grumpy (0)	Satisfied (7)	Grumpy (0)

Base Satisfied Sum = 1 + 1 + 1 + 7 = 10

Approach 1: Prefix Sum Array Strategy

This approach explicitly calculates a gain array to isolate the potential satisfied customers we can "rescue" if the owner uses the technique during those minutes. A prefix sum array is built on top of this gain array to quickly query any window of size *minutes* in *O(1)* time.

Algorithmic Steps

- **Step 1 (Base Case Answer):** Compute the baseline satisfied customers when the owner is naturally calm (`!grumpy[i]`).
- **Step 2 (Gain Array Generation):** Build an auxiliary gain array where `gain[i] = grumpy[i] == 1 ? customers[i] : 0;`
- **Step 3 (Prefix Sum & Window Evaluation):** Accumulate prefix sums of the gain array. Iterate through all valid sub-arrays of size *minutes* and find the maximum rescue value using the formula *prefix[r + 1] - prefix[l]*.

TIME COMPLEXITY

$O(n)$

SPACE COMPLEXITY

$O(n)$

Implementation Source Code

```

class Solution {
public:
    int maxSatisfied(vector& customers, vector& grumpy, int minutes) {

        int base=0;
        int n=customers.size();
        vector gain(n);
        for(int i=0;i < minutes;i++) prefix(n+1,0);
        for(int i=0;i < n;i++)
    
```

Approach 2: Optimal Sliding Window Strategy

Instead of creating dedicated auxiliary arrays for gains and prefix sums, this approach optimizes space by using a single array to track the net gain of the sliding window.

Algorithmic Steps

- **Step 1 (Baseline Score):** Linearly count all naturally satisfied customers across the entire timeline to establish the baseline score.
- **Step 2 (Initial Window Evaluation):** Compute the gain for the first window spanning from index 0 to minutes - 1.
- **Step 3 (Sliding Window Maintenance):** Shift the window rightward from index minutes to n - 1. For each position, calculate the net gain by comparing the current window's performance against the baseline.

TIME COMPLEXITY

$O(n)$

SPACE COMPLEXITY

$O(1)$

Implementation Source Code

```
class Solution {  
public:  
    int maxSatisfied(vector& customers, vector& grumpy, int minutes) {  
        int n=customers.size();  
        int score=0;  
        for(int i=0;i
```